

Federated Fine-Tuning of Large Models: Optimization Trade-offs and Privacy Challenges

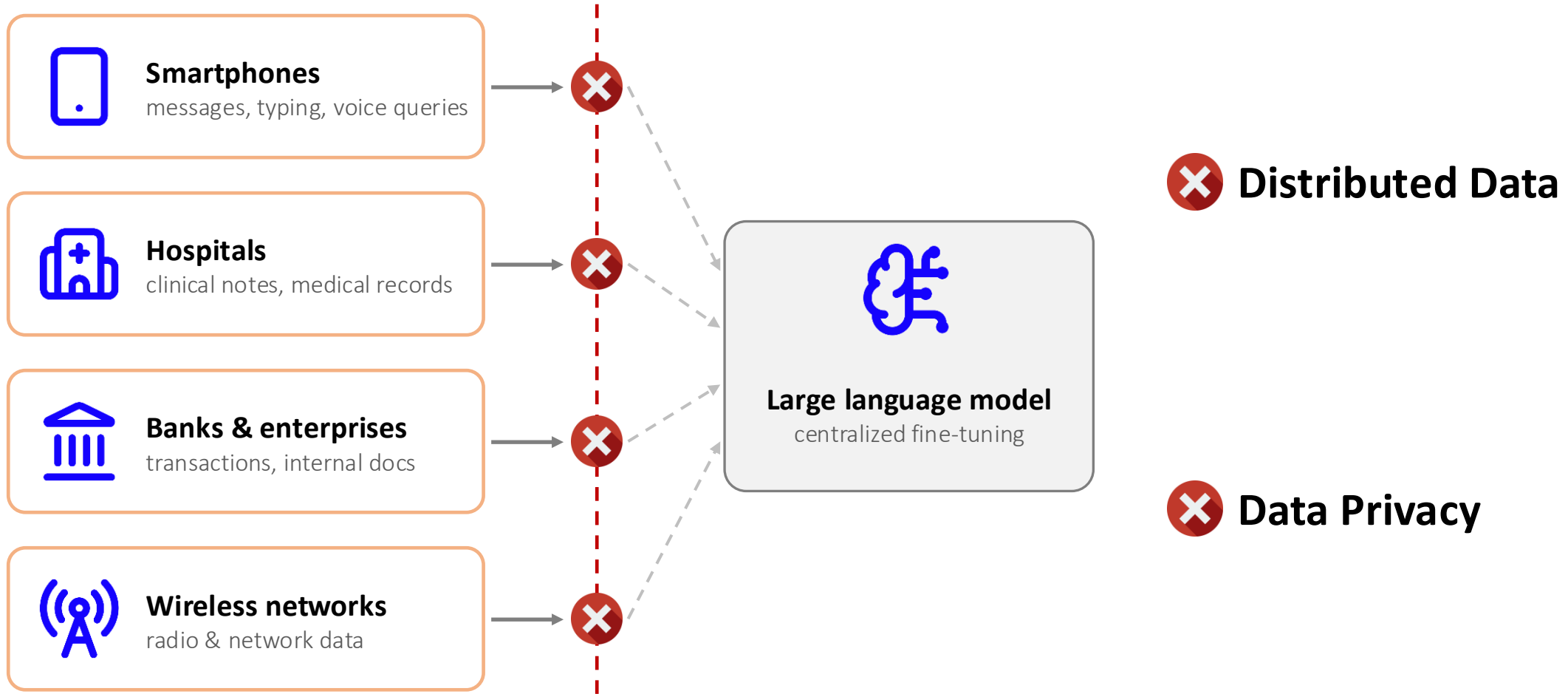
Ramy E. Ali

Samsung

October 2nd, 2026

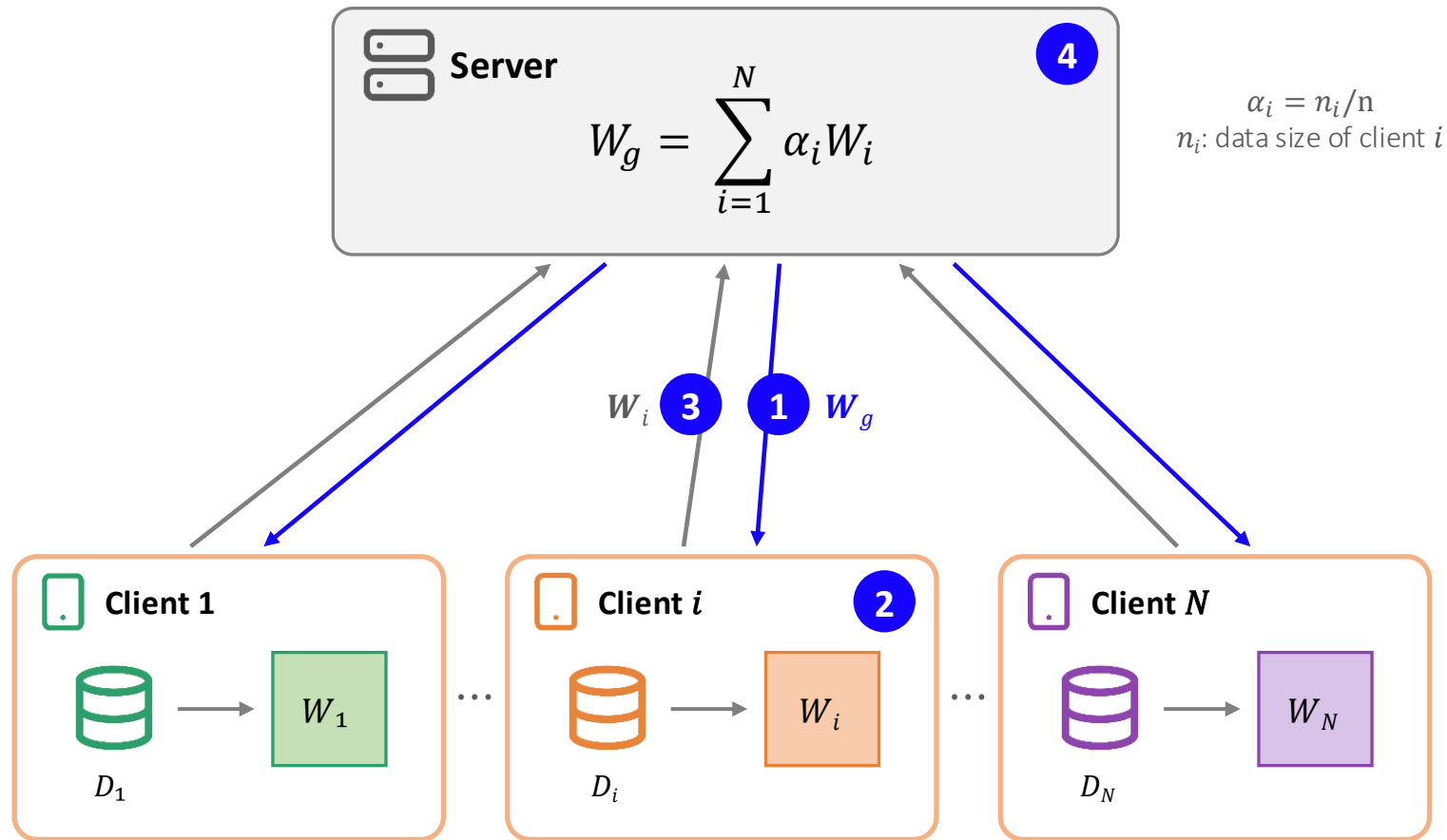
Joint work with CoRE-AI Lab (Juseok Jeon, Jinhyun So, ...), DGIST, South Korea

Distributed Data in the Real World



Privacy regulations (e.g., General Data Protection Regulation (GDPR), Health Insurance Portability and Accountability Act (HIPAA)), data ownership, and bandwidth prevent pooling the data

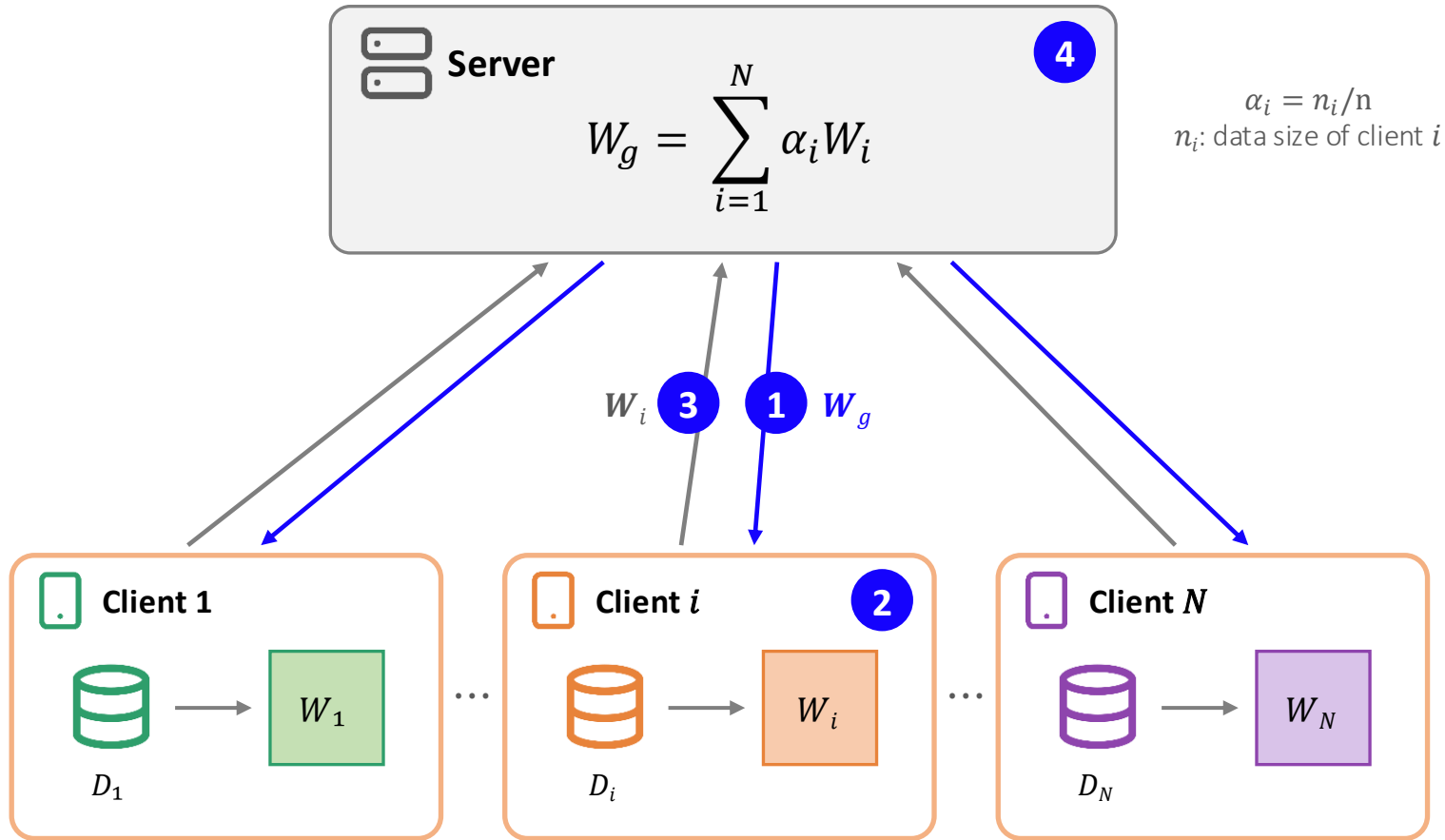
Federated Learning



- ✓ **Distributed Data**
- ✓ **Increased Data Privacy**

① Server sends W_g ② Clients train on local data ③ Clients upload W_i ④ Server averages, repeat

Federated Learning



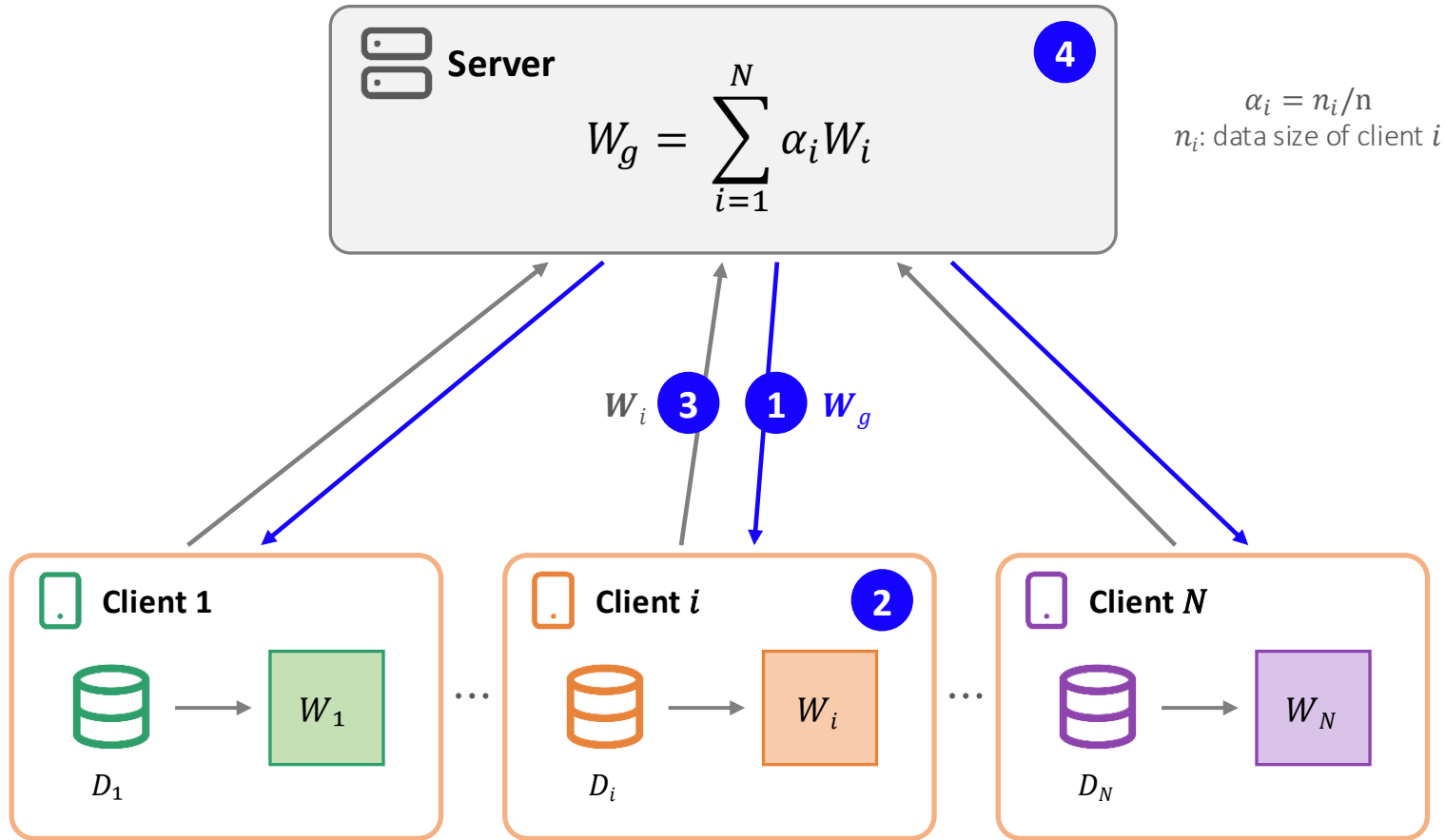
- ✓ **Distributed Data**
- ✓ **Increased Data Privacy**

- ✗ **Computation Cost**
every client trains the full model

- ✗ **Communication Cost**
every round, each client downloads W_g and uploads W_i ; the full model (≈ 16 GB for 8B parameters)

① Server sends W_g ② Clients train on local data ③ Clients upload W_i ④ Server averages, repeat

Federated Learning



- ✓ **Distributed Data**
- ✓ **Increased Data Privacy**

✗ **Computation Cost**
every client trains the full model

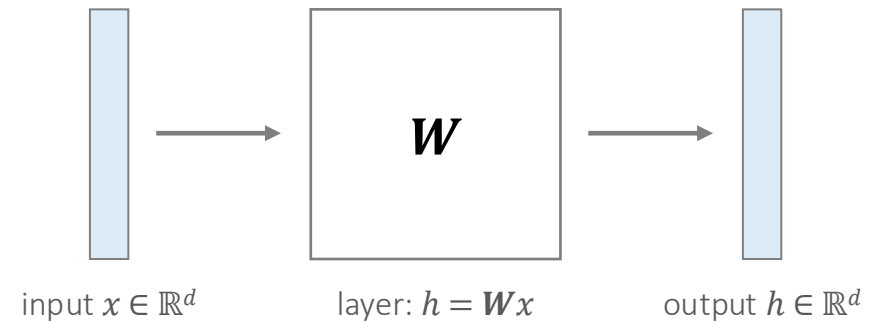
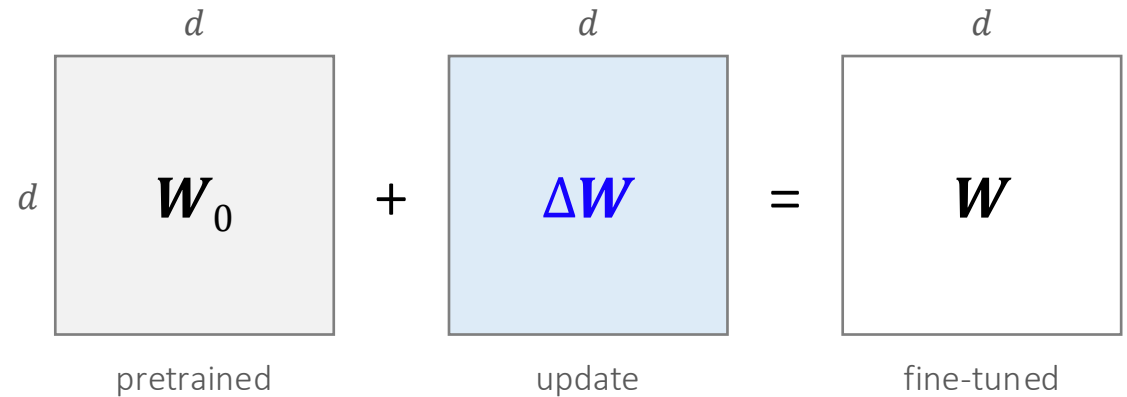
✗ **Communication Cost**
every round, each client downloads W_g and uploads W_i ; the full model (≈ 16 GB for 8B parameters)

① Server sends W_g ② Clients train on local data ③ Clients upload W_i ④ Server averages, repeat

Idea: Only fine-tune instead of full model training

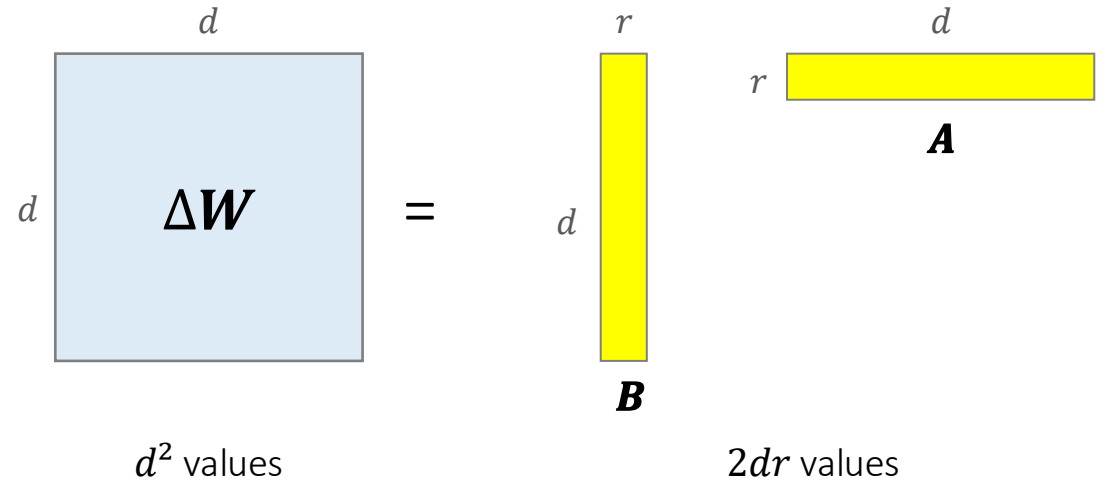
Fine-Tuning a Pretrained Model

- Each layer: $h = \mathbf{W}x$, weights $\mathbf{W} \in \mathbb{R}^{d \times d}$
 - d = hidden size (4096 in Llama 3-8B)
 - attention / MLP projections map $d \rightarrow d$ ($\mathbf{W}$ is square)
- Fine-tuning adapts $\mathbf{W}_0$: $\mathbf{W} = \mathbf{W}_0 + \Delta\mathbf{W}$
 - the update $\Delta\mathbf{W} \in \mathbb{R}^{d \times d}$ has the same shape
 - full fine-tuning learns all d^2 entries, every layer



Why LoRA? Learn a Low-Rank Update

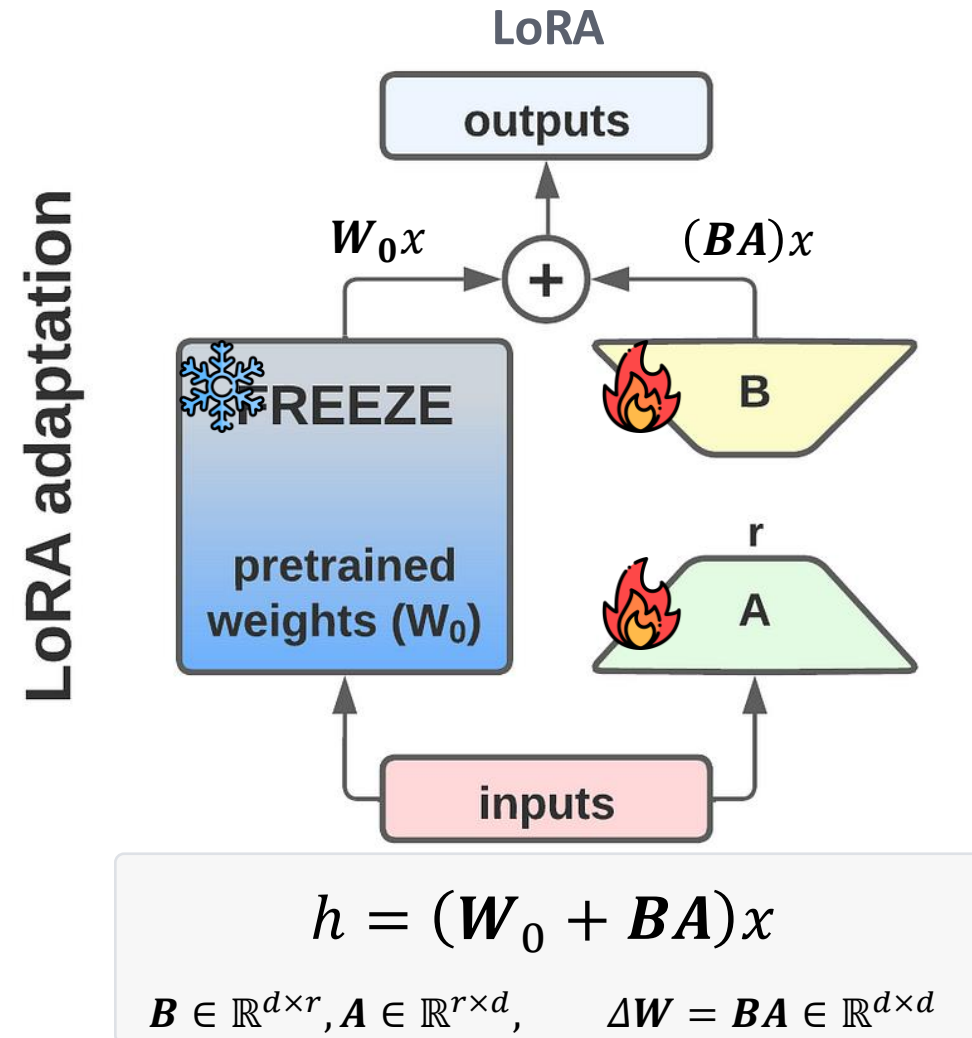
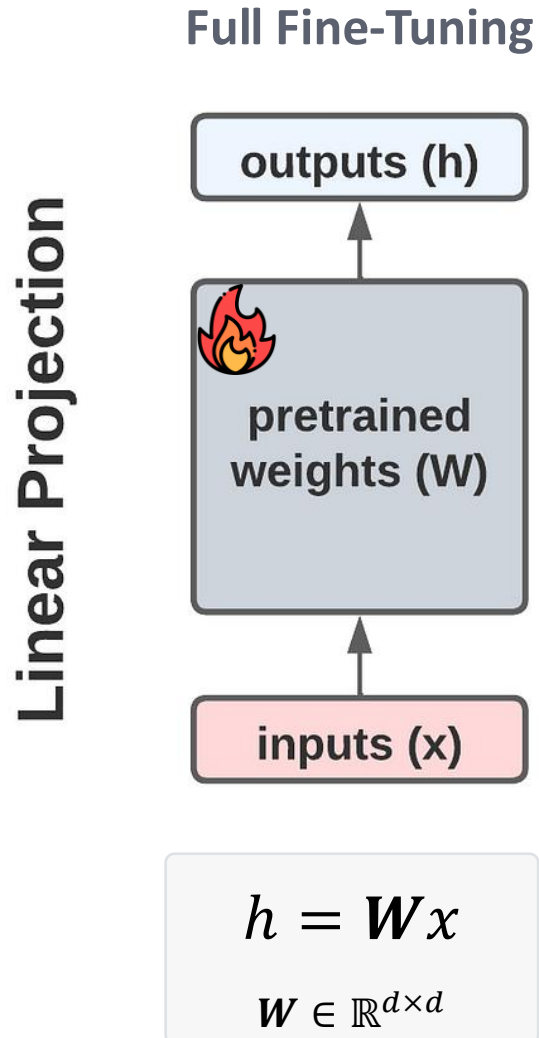
- Fine-tuning: $\mathbf{W} = \mathbf{W}_0 + \Delta\mathbf{W}$, where $\Delta\mathbf{W}$
- Observation: $\Delta\mathbf{W}$ has low intrinsic rank
- LoRA writes $\Delta\mathbf{W}$ as a product of two thin matrices
 - $\Delta\mathbf{W} = \mathbf{BA}$, $\mathbf{B} \in \mathbb{R}^{d \times r}$, $\mathbf{A} \in \mathbb{R}^{r \times d}$, rank $r \ll d$
- Freeze $\mathbf{W}_0$; train only $\mathbf{B}$ and $\mathbf{A}$
 - $2dr$ values instead of d^2 per matrix
 - Merge $\mathbf{BA}$ into $\mathbf{W}_0$ after training: no extra latency



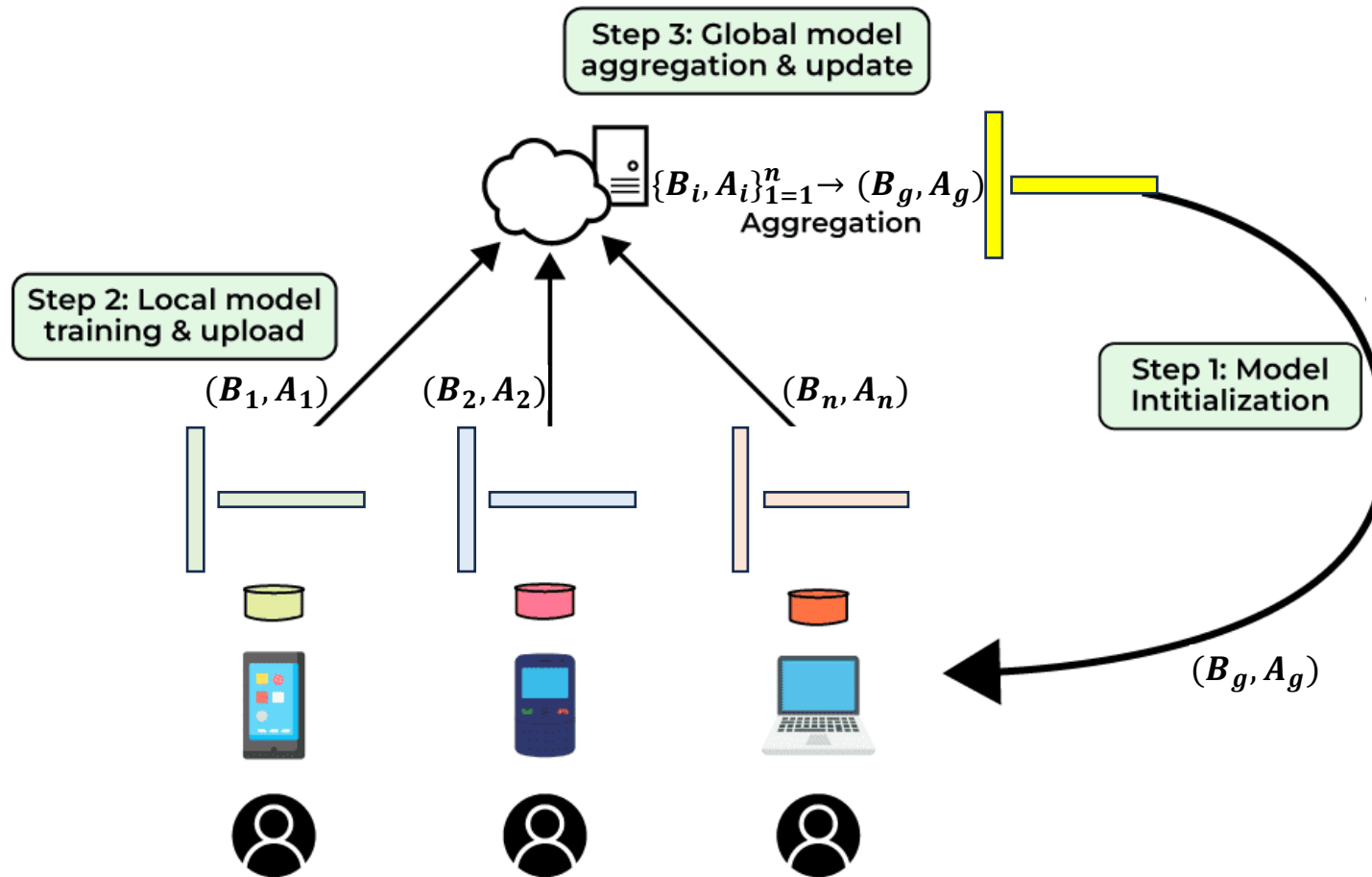
256×

fewer trainable parameters per matrix
($d = 4096, r = 8$: 16.8M $\rightarrow$ 65.5K)

How LoRA Works?

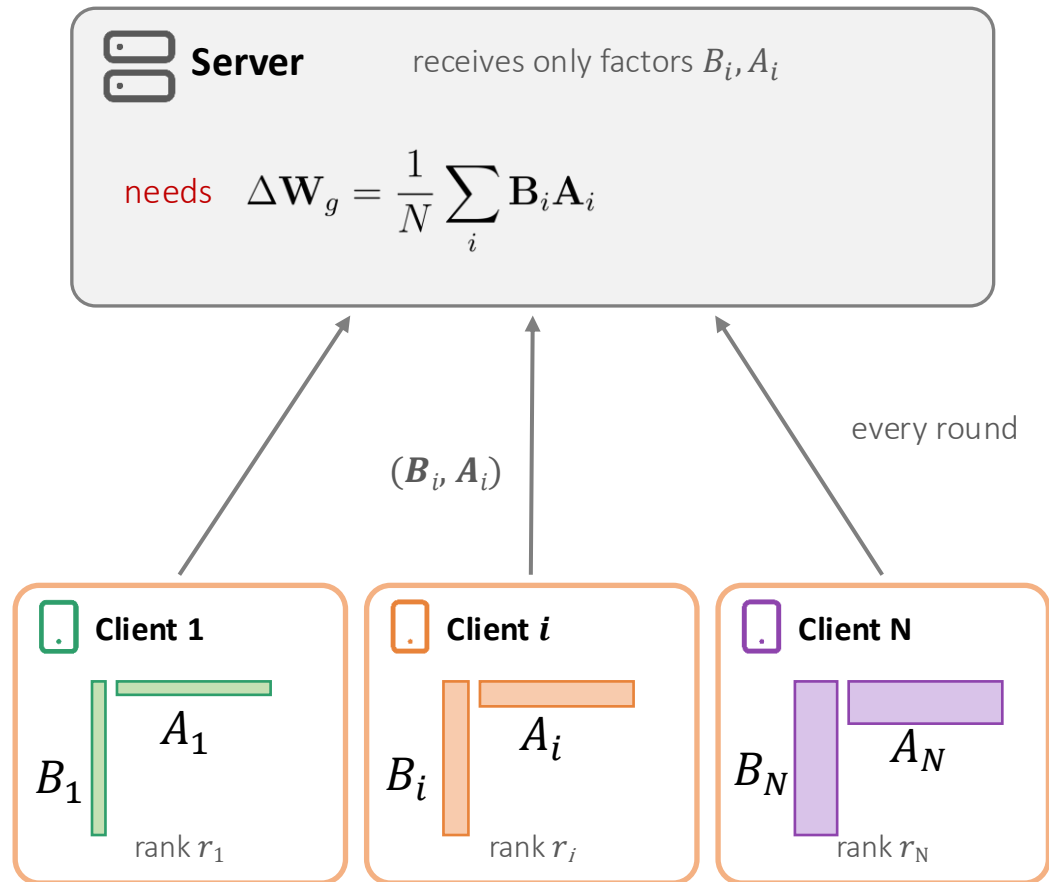


How to leverage LoRA in FL? Federated LoRA



- ✓ **Distributed Data**
- ✓ **Increased Data Privacy**
- ✓ **Computation Cost**
- ✓ **Communication Cost**

Why Is LoRA Challenging in Federated Learning?



1 Aggregation

Averaging the factors is not averaging the updates

$$\left(\frac{1}{N} \sum_i \mathbf{B}_i\right) \left(\frac{1}{N} \sum_i \mathbf{A}_i\right) \neq \frac{1}{N} \sum_i \mathbf{B}_i \mathbf{A}_i$$

2 Local continuity

Exact averaging + re-factorizing replaces each client's own factors

3 Heterogeneous ranks

Clients with different r_i send factors of different shapes

4 Privacy

Server sees every $(\mathbf{B}_i, \mathbf{A}_i)$; secure aggregation only sums

These four challenges define the state of the art and the [research gap](#) (next)

Challenge 1: Averaging Factors $\neq$ Averaging Updates

- 1 The server needs the average of the products, but FedIT averages each factor

Ideal update

$$\Delta \mathbf{W}_{\text{ideal}}^{(t)} = \frac{1}{N} \sum_{i=1}^N \mathbf{B}_i^{(t)} \mathbf{A}_i^{(t)}$$

FedIT

$$\mathbf{B}_g^{(t)} \mathbf{A}_g^{(t)} = \left(\frac{1}{N} \sum_i \mathbf{B}_i^{(t)} \right) \left(\frac{1}{N} \sum_j \mathbf{A}_j^{(t)} \right) = \frac{1}{N^2} \sum_i \sum_j \mathbf{B}_i^{(t)} \mathbf{A}_j^{(t)}$$

Aggregation error

$$\Delta \mathbf{W}_{\text{ideal}} - \mathbf{B}_g \mathbf{A}_g = \frac{1}{2N^2} \sum_i \sum_j (\mathbf{B}_i - \mathbf{B}_j)(\mathbf{A}_i - \mathbf{A}_j)$$

- ✓ Zero only if all clients share the same $\mathbf{B}$ or the same $\mathbf{A}$; grows as their factors diverge (non-IID data)

$\mathbf{B}_g \mathbf{A}_g$ mixes all N^2 products ($N = 3$)

	$\mathbf{A}_1$	$\mathbf{A}_2$	$\mathbf{A}_3$
$\mathbf{B}_1$	$\mathbf{B}_1 \mathbf{A}_1$	$\mathbf{B}_1 \mathbf{A}_2$	$\mathbf{B}_1 \mathbf{A}_3$
$\mathbf{B}_2$	$\mathbf{B}_2 \mathbf{A}_1$	$\mathbf{B}_2 \mathbf{A}_2$	$\mathbf{B}_2 \mathbf{A}_3$
$\mathbf{B}_3$	$\mathbf{B}_3 \mathbf{A}_1$	$\mathbf{B}_3 \mathbf{A}_2$	$\mathbf{B}_3 \mathbf{A}_3$

 trained by a client, weight $1/N^2$ instead of $1/N$

 cross terms $\mathbf{B}_i \mathbf{A}_j$, $i \neq j$: no client trained them

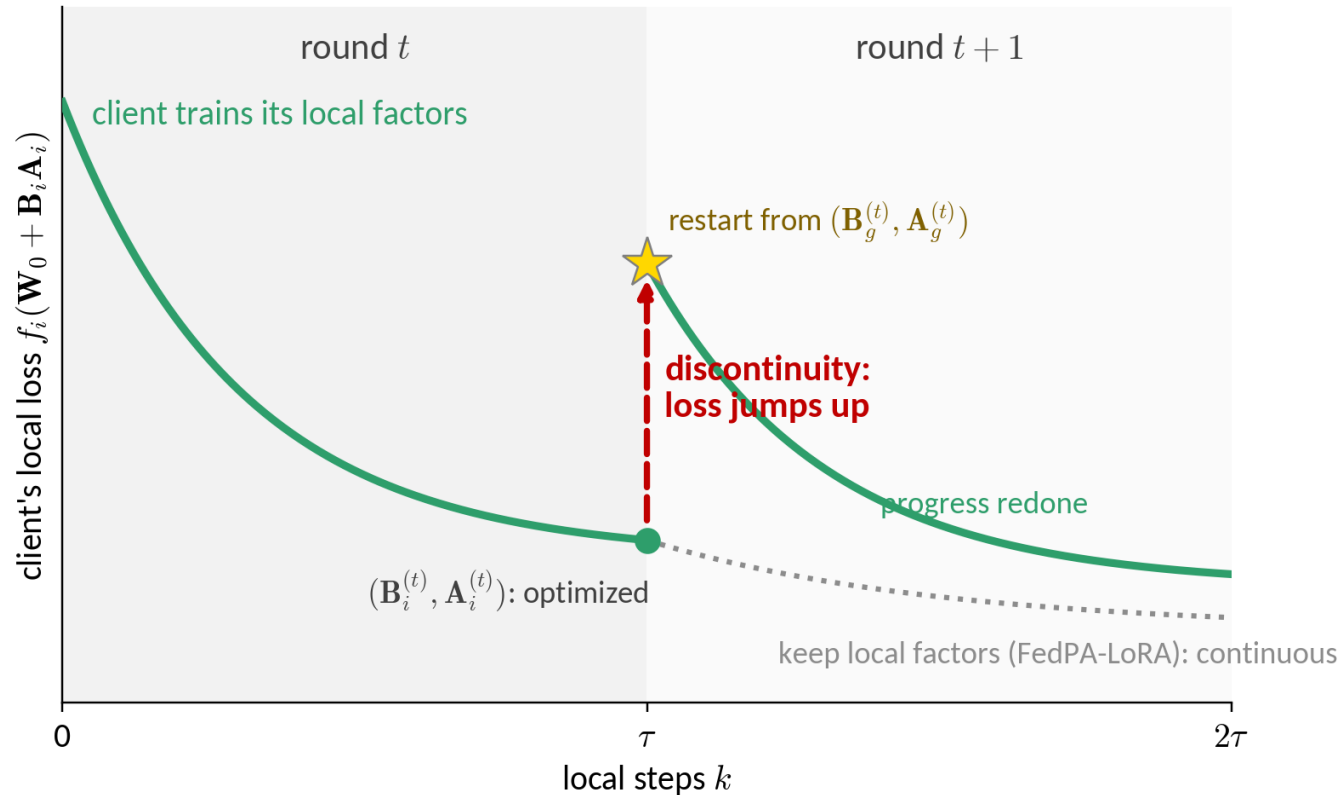
FedIT = FedAvg on the factors: it averages $\mathbf{B}$ and $\mathbf{A}$ separately, like ordinary weights

The adapter is then their product $\mathbf{B}_g \mathbf{A}_g$, the server never forms the products $\mathbf{B}_i \mathbf{A}_i$

But average-of-products $\neq$ product-of-averages

Challenge 2: Local Continuity

- 2 The client was optimizing its own factors; replacing them undoes progress



At the round boundary

$$(\mathbf{B}_i^{(t+1,0)}, \mathbf{A}_i^{(t+1,0)}) = (\mathbf{B}_g^{(t)}, \mathbf{A}_g^{(t)})$$

$$\neq (\mathbf{B}_i^{(t)}, \mathbf{A}_i^{(t)})$$

- ✓ The locally optimized factors are discarded
- ✓ Re-factorizing changes the factors even where the product agrees: $\mathbf{B}\mathbf{A} = (\mathbf{B}\mathbf{Q})(\mathbf{Q}^{-1}\mathbf{A})$

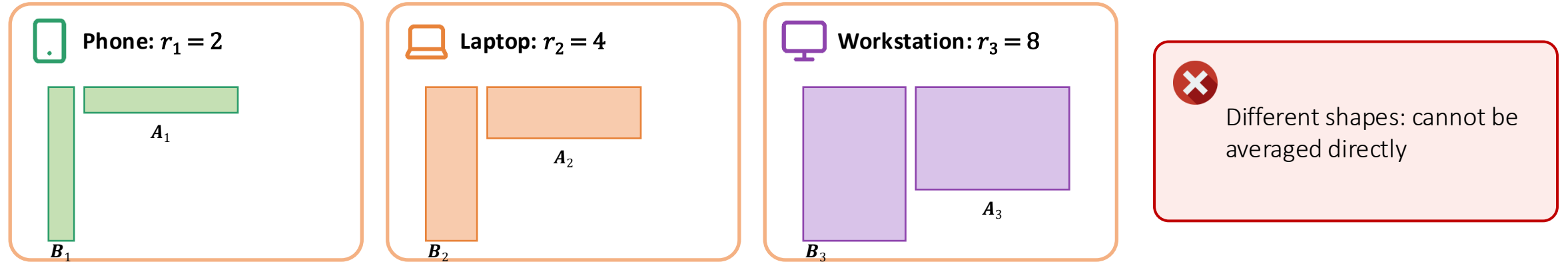
Measured in the paper by

$$E_{\text{init}}^{(t)}(\mathbf{Y}) = \frac{1}{N} \sum_{i=1}^N \frac{\|\mathbf{Y}_i^{(t+1,0)} - \mathbf{Y}_i^{(t)}\|_F}{\|\mathbf{Y}_i^{(t+1,0)}\|_F + \varepsilon}$$

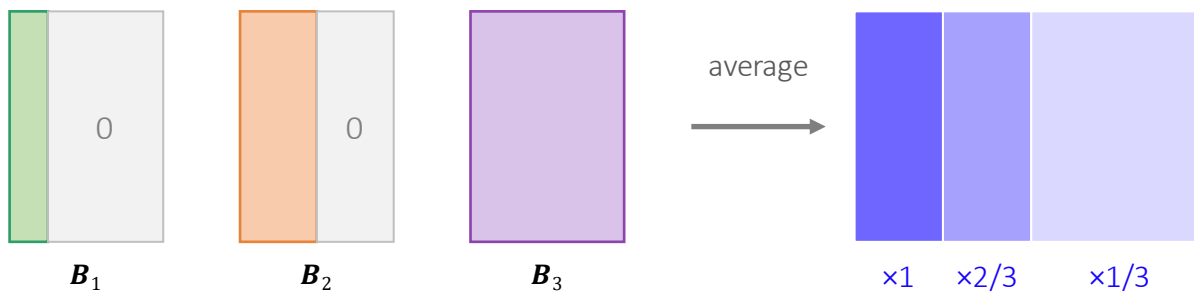
Every round the client partly redoes its progress: [factor-level initialization mismatch](#)

Challenge 3: Heterogeneous Ranks

3 Client i trains $\mathbf{B}_i \in \mathbb{R}^{d \times r_i}$, $\mathbf{A}_i \in \mathbb{R}^{r_i \times d}$, with rank r_i set by its memory and bandwidth



A common fix: zero-pad every factor to the largest rank



- ✓ Shapes now match, but directions that only large-rank clients learned are averaged with zeros: shrunk to $1/3$ here
- ✓ The factor-averaging error of Challenge 1 remains

Challenge 4: Privacy

4 Shared factors can leak data, and the standard defense does not fit LoRA

The LoRA attack (DAGER): recover input tokens from the shared factor gradients

① Setup — what the client shares

$$\mathbf{h} = \mathbf{W}_0 \mathbf{x} + \mathbf{B}(\mathbf{A}\mathbf{x})$$

$\mathbf{W}_0$ frozen; $\mathbf{x}_t$ = token embedding
shared updates = gradients $\nabla_{\mathbf{A}}\mathcal{L}, \nabla_{\mathbf{B}}\mathcal{L}$

② The leak: $\nabla_{\mathbf{A}}\mathcal{L}$ is low-rank (only r rows)

$$\nabla_{\mathbf{A}}\mathcal{L} = \sum_t (\mathbf{B}^\top \delta_t) \mathbf{x}_t^\top \in \mathbb{R}^{r \times d}$$

row space of $\nabla_{\mathbf{A}}\mathcal{L}$ = span of the inputs $\mathbf{x}_t$ = the input token embeddings

③ Recover the tokens — filter the vocabulary

for each token w : $\mathbf{e}_w \in \text{rowspace}(\nabla_{\mathbf{A}}\mathcal{L})$?

matches = the client's input tokens · exact when $\#\text{tokens} \leq r$ · order by greedy search

Uses: the shared $\nabla_{\mathbf{A}}\mathcal{L}$ ($r \times d$) + the public embedding table $\mathbf{E}$. $\mathbf{B}, \mathbf{A}$ shape the gradient; the row-space test needs only $\nabla_{\mathbf{A}}\mathcal{L}$.

The standard defense: secure aggregation

 Server

gets $\sum_i \mathbf{B}_i, \quad \sum_i \mathbf{A}_i$

needs $\sum_i \mathbf{B}_i \mathbf{A}_i$



- ✓ Masks cancel only in sums
- ✓ LoRA aggregation multiplies factors: not a sum

Research Approach: Exploit the Low-Rank Structure



Optimization

- ✓ Aggregate in product space
- ✓ Keep local factors



Privacy

- ✓ Attacks on low-rank updates
- ✓ Information-theoretic and differential privacy (DP) measures



Security

- ✓ Robust and verifiable aggregation

Tools: optimization theory · information and coding theory · cryptography · large-scale LLM experiments

Main technical challenges

- Bilinear aggregation vs. linear secure aggregation
- Nonlinear reconstruction (QR / SVD)
- Persistent local factors across rounds
- Heterogeneous data and ranks
- Scale: billions of parameters

Initial investigation I: aggregation (preliminary)

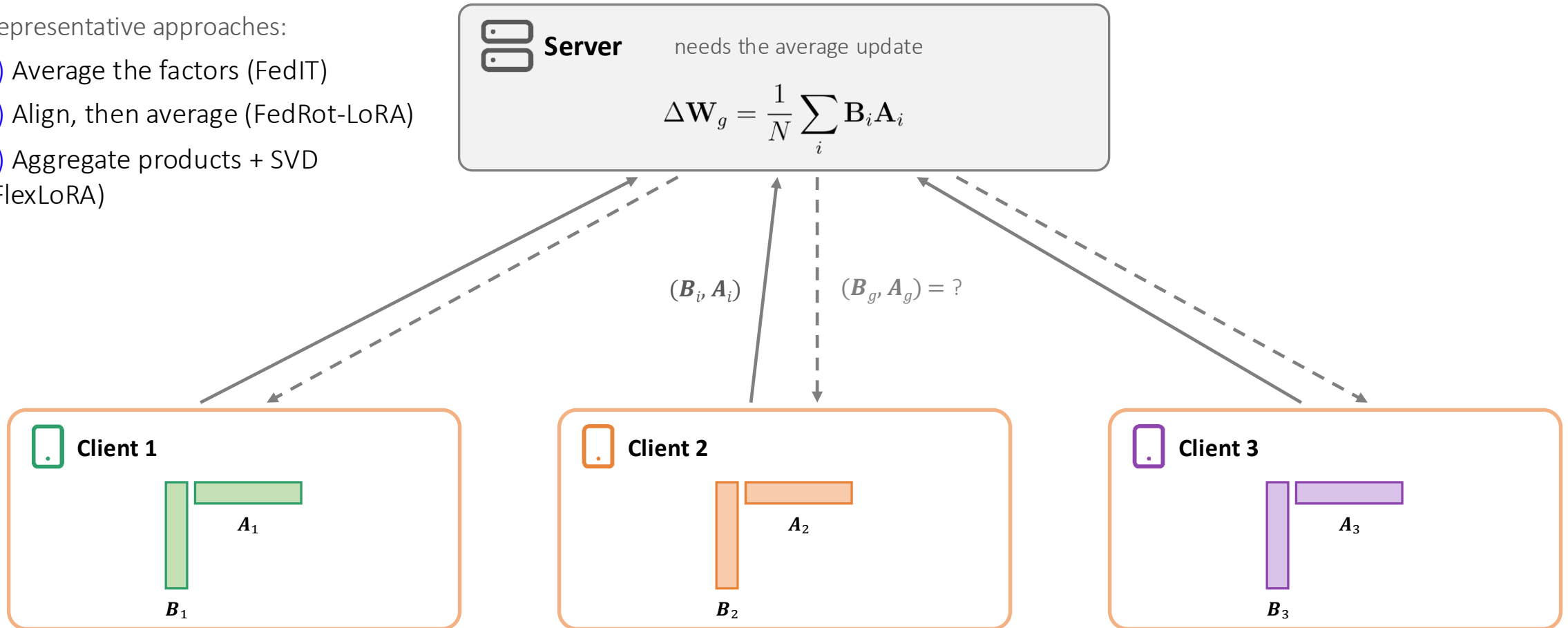
Next investigation II: privacy

First two years: plan, checkpoints, risks

How Does the Server Aggregate?

Representative approaches:

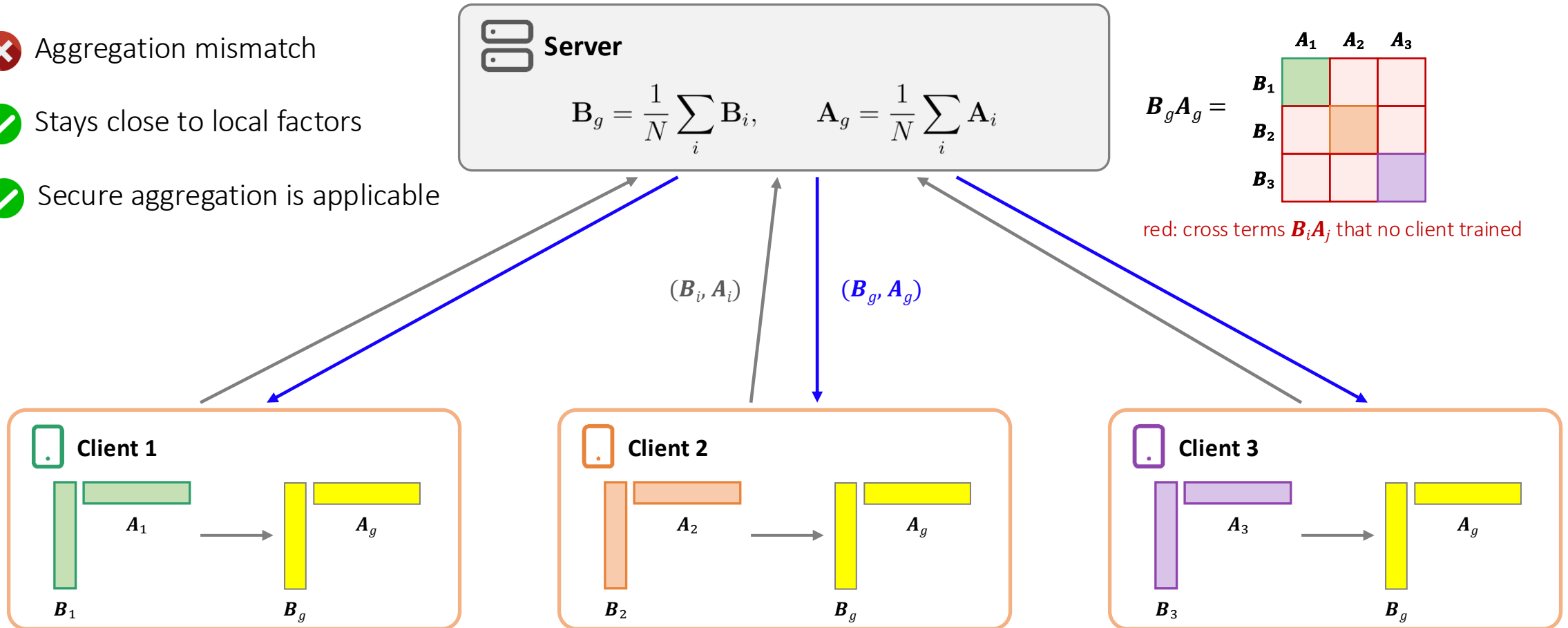
- 1) Average the factors (FedIT)
- 2) Align, then average (FedRot-LoRA)
- 3) Aggregate products + SVD (FlexLoRA)



Each client trains its own low-rank factors on local data

Option 1: Average the Factors (FedIT = Federated Instruction Tuning)

- ✗ Aggregation mismatch
- ✓ Stays close to local factors
- ✓ Secure aggregation is applicable



Clients restart from the averaged factors

FedIT = FedAvg applied to LoRA: average $\mathbf{B}$ and $\mathbf{A}$ like ordinary weights

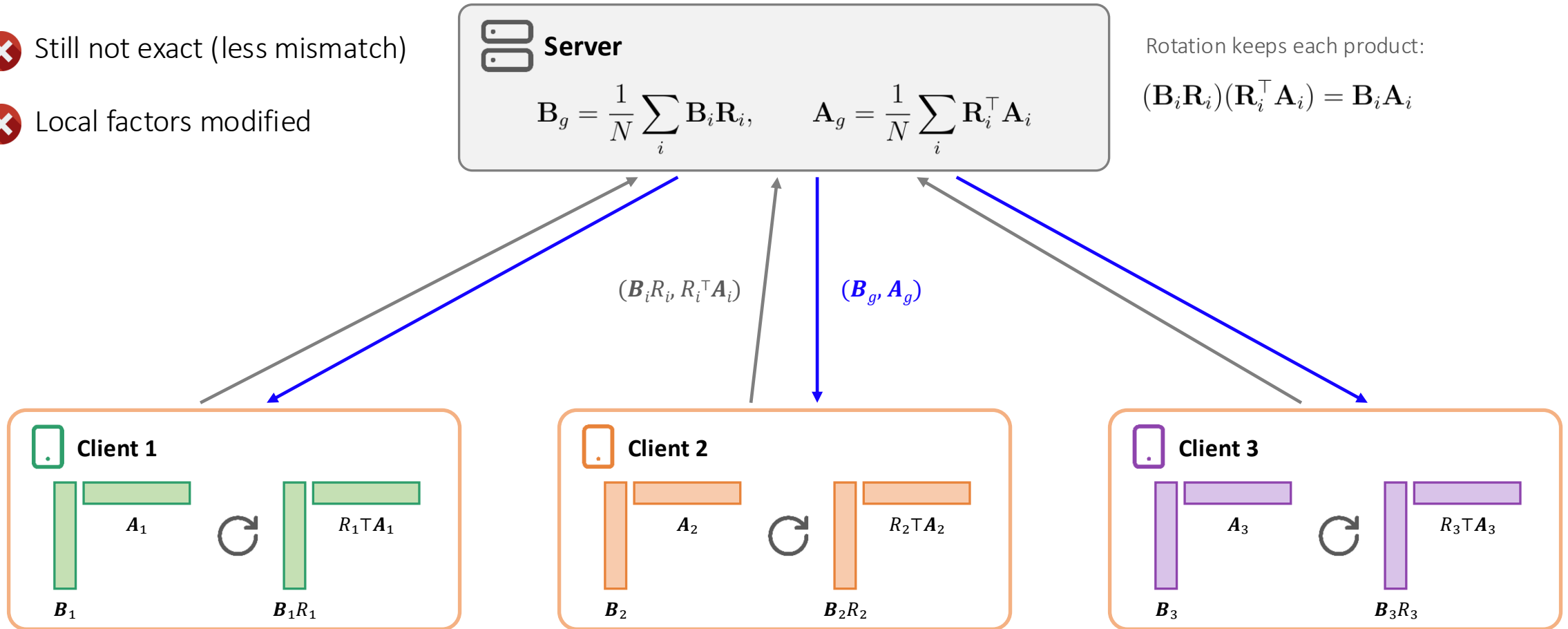
Simple, and communication-cheap

Goal is simplicity + low comms

Option 2: Align, then Average (FedRot-LoRA)

✗ Still not exact (less mismatch)

✗ Local factors modified



Clients rotate their factors to align them before averaging, then restart from $(\mathbf{B}_g, \mathbf{A}_g)$

How FedRot-LoRA Aligns Factors Before Averaging

2 Rotate each client's factors into a common frame, then average

A rotation keeps the product, changes the factors

$$\tilde{\mathbf{B}}_i = \mathbf{B}_i \mathbf{R}_i, \quad \tilde{\mathbf{A}}_i = \mathbf{R}_i^\top \mathbf{A}_i, \quad \mathbf{R}_i^\top \mathbf{R}_i = \mathbf{I}_r$$

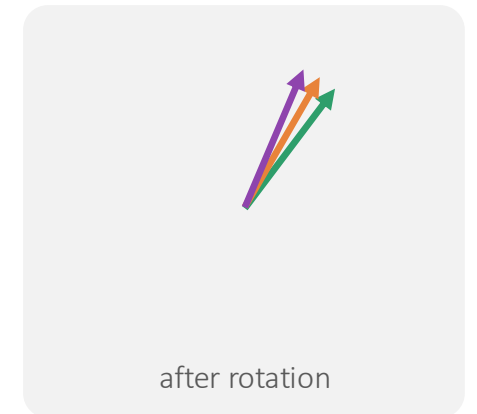
$$\tilde{\mathbf{B}}_i \tilde{\mathbf{A}}_i = \mathbf{B}_i \mathbf{R}_i \mathbf{R}_i^\top \mathbf{A}_i = \mathbf{B}_i \mathbf{A}_i$$

- ✓ One rotation $\mathbf{R}_i$ transforms both; we fix it by aligning $\mathbf{B}$ to $\mathbf{B}_g$ only — $\mathbf{A}$ then carries along ($\mathbf{R}_i^\top \mathbf{A}_i$)
- ✓ $\mathbf{R}_i^\top \mathbf{R}_i = \mathbf{I}$: the rotation preserves each client's own product (lossless, within-client)

Then the server averages the rotated factors across clients

- still $\neq$ averaging the products $\rightarrow$ not exact (cross-terms shrink, don't vanish)
- each client's factors are replaced by the averaged ($\mathbf{B}_g, \mathbf{A}_g$)

Intuition: factorizations point in different frames



How is each $\mathbf{R}_i$ found?

Align each client's $\mathbf{B}_i$ to the global $\mathbf{B}_g$ (orthogonal Procrustes):

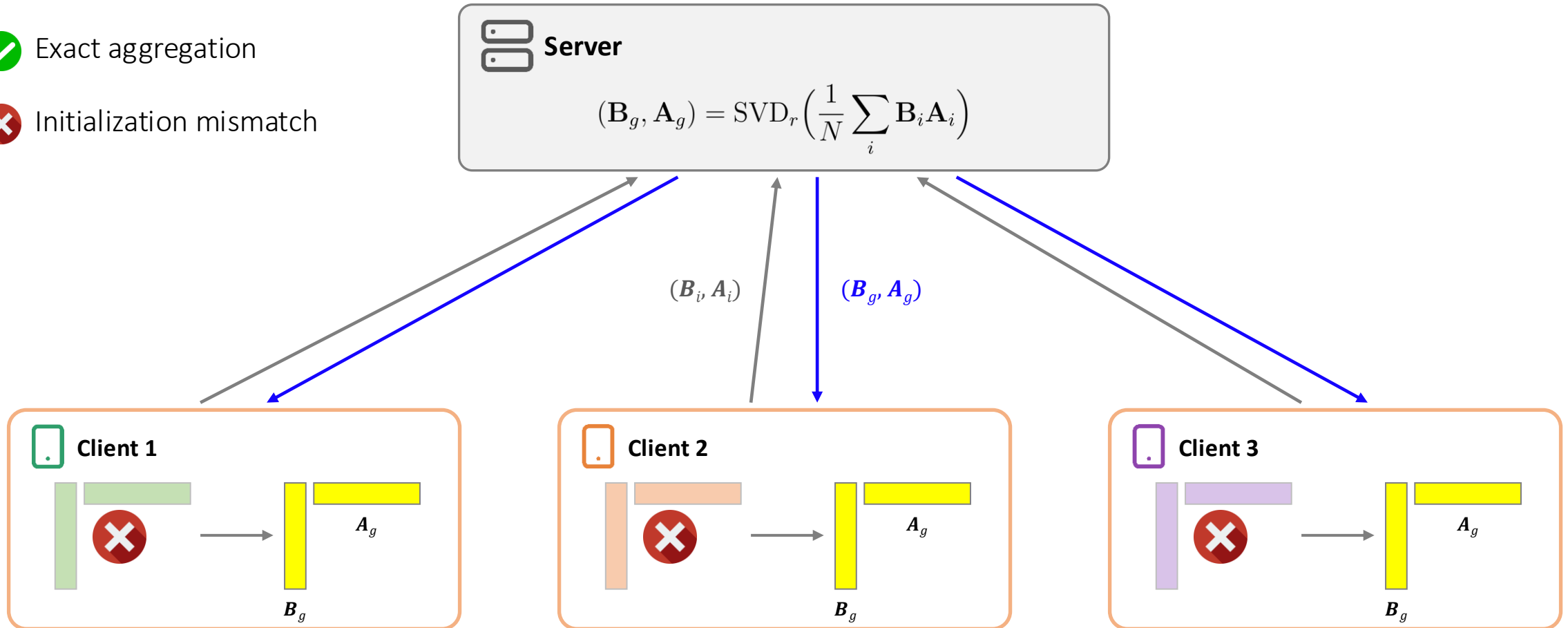
$$\min_{\mathbf{R}_i^\top \mathbf{R}_i = \mathbf{I}} \|\mathbf{B}_i \mathbf{R}_i - \mathbf{B}_g\|_F$$

then take the SVD, read off $\mathbf{R}_i$:

$$\mathbf{B}_i^\top \mathbf{B}_g = \mathbf{U} \Sigma \mathbf{V}^\top \Rightarrow \mathbf{R}_i = \mathbf{U} \mathbf{V}^\top$$

Option 3: Aggregate Products, then SVD (FlexLoRA)

- ✓ Exact aggregation
- ✗ Initialization mismatch



Clients discard their own factors and restart from new SVD factors every round

What the SVD Does on the Server (FlexLoRA)

3 The average is exact; the SVD only changes its shape — a rank- Nr matrix back into a rank- r adapter

1. Form the exact average

$$\Delta \mathbf{W}_{\text{ideal}} = \frac{1}{N} \sum_i \mathbf{B}_i \mathbf{A}_i$$

server has each $(\mathbf{B}_i, \mathbf{A}_i) \rightarrow$ sums the true products

2. Truncated SVD $\rightarrow$ best rank r

$$\Delta \mathbf{W}_{\text{ideal}} \approx \mathbf{U}_r \Sigma_r \mathbf{V}_r^\top$$

keep the top r singular directions

3. Read off factors

$$\mathbf{B}_g = \mathbf{U}_r \Sigma_r, \mathbf{A}_g = \mathbf{V}_r^\top$$

rank- r global adapter

Why SVD?

- ✓ The exact average is a dense $d \times d$ matrix of rank up to Nr (3 clients $\times$ rank 8 $\rightarrow$ rank 24)
- ✓ Clients need a rank- r pair $(\mathbf{B}_g, \mathbf{A}_g)$, not a dense or rank- Nr matrix
- ✓ SVD gives the best rank- r factored form (Eckart–Young): smallest possible loss
- ✓ Keeping the full rank Nr is exact, but it grows every round

The catch

Aggregation is exact (no FedIT cross terms); only this compression is approximate. And SVD factors are new (not unique):

$$\mathbf{B}_g \mathbf{A}_g = (\mathbf{B}_g \mathbf{Q})(\mathbf{Q}^{-1} \mathbf{A}_g)$$

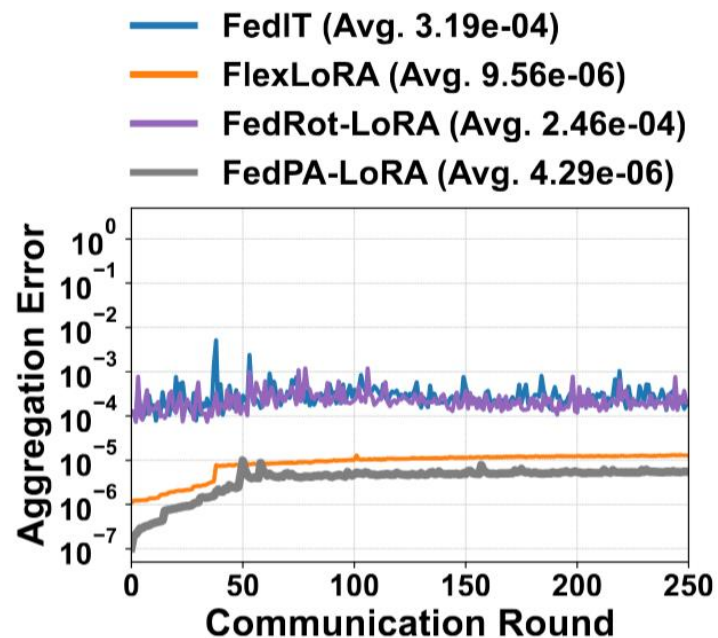
Clients are reset $\rightarrow$ initialization mismatch

Motivating Example

Empirical observation: RoBERTa-Large on MNLI, 3 clients, $r = 4$

1 Aggregation Error

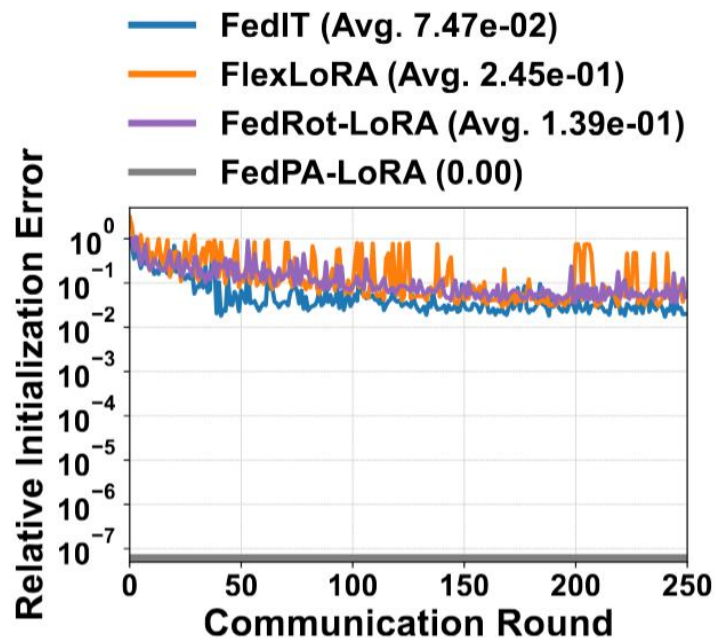
$$\mathbb{E}_{\text{agg}}^{(t)} = \left\| \mathbf{B}_g^{(t)} \mathbf{A}_g^{(t)} - \Delta \mathbf{W}_{\text{ideal}}^{(t)} \right\|_{\text{F}}$$



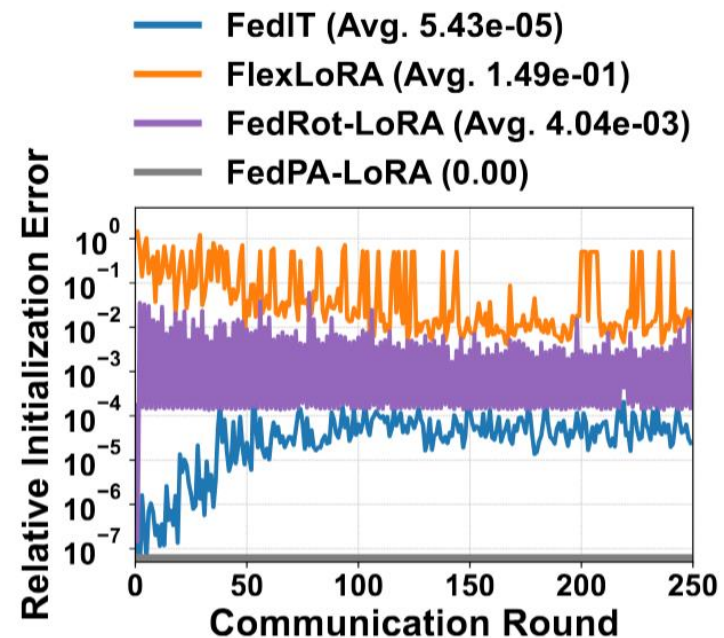
(a) Aggregation Error

2 Factor-Level Initialization Error

$$\mathbb{E}_{\text{init}}^{(t)}(\mathbf{Y}) = \frac{1}{N} \sum_{i=1}^N \frac{\left\| \mathbf{Y}_i^{(t+1,0)} - \mathbf{Y}_i^{(t)} \right\|_{\text{F}}}{\left\| \mathbf{Y}_i^{(t+1,0)} \right\|_{\text{F}} + \varepsilon}, \quad \mathbf{Y} \in \{\mathbf{B}, \mathbf{A}\}$$



(b) Initialization Error of $\mathbf{B}$



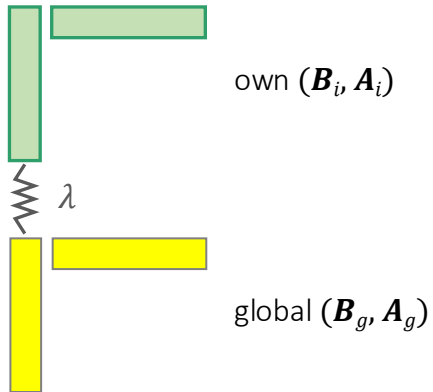
(c) Initialization Error of $\mathbf{A}$

Keeping Local Factors: What to Pull Toward?



Server: aggregates the products exactly and sends $(\mathbf{B}_g, \mathbf{A}_g)$ to every client

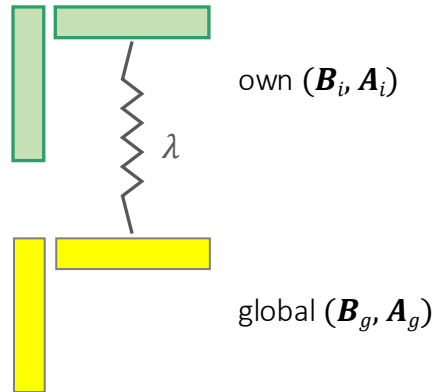
Pull $\mathbf{B}$



$$\|\mathbf{B}_i - \mathbf{B}_g\|_F^2$$

MNLI 0.875

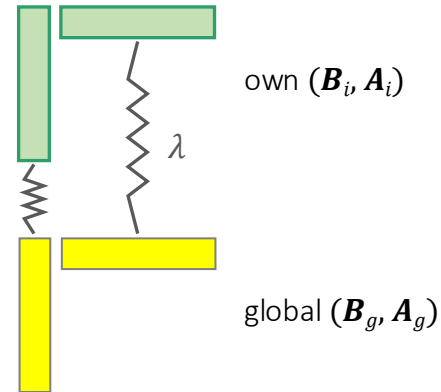
Pull $\mathbf{A}$



$$\|\mathbf{A}_i - \mathbf{A}_g\|_F^2$$

MNLI 0.879

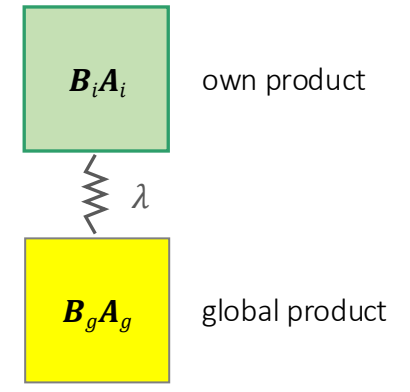
Pull $\mathbf{B}$ and $\mathbf{A}$



$$\|\mathbf{B}_i - \mathbf{B}_g\|_F^2 + \|\mathbf{A}_i - \mathbf{A}_g\|_F^2$$

MNLI 0.896

Pull the product $\mathbf{BA}$



$$\|\mathbf{B}_i \mathbf{A}_i - \mathbf{B}_g \mathbf{A}_g\|_F^2$$

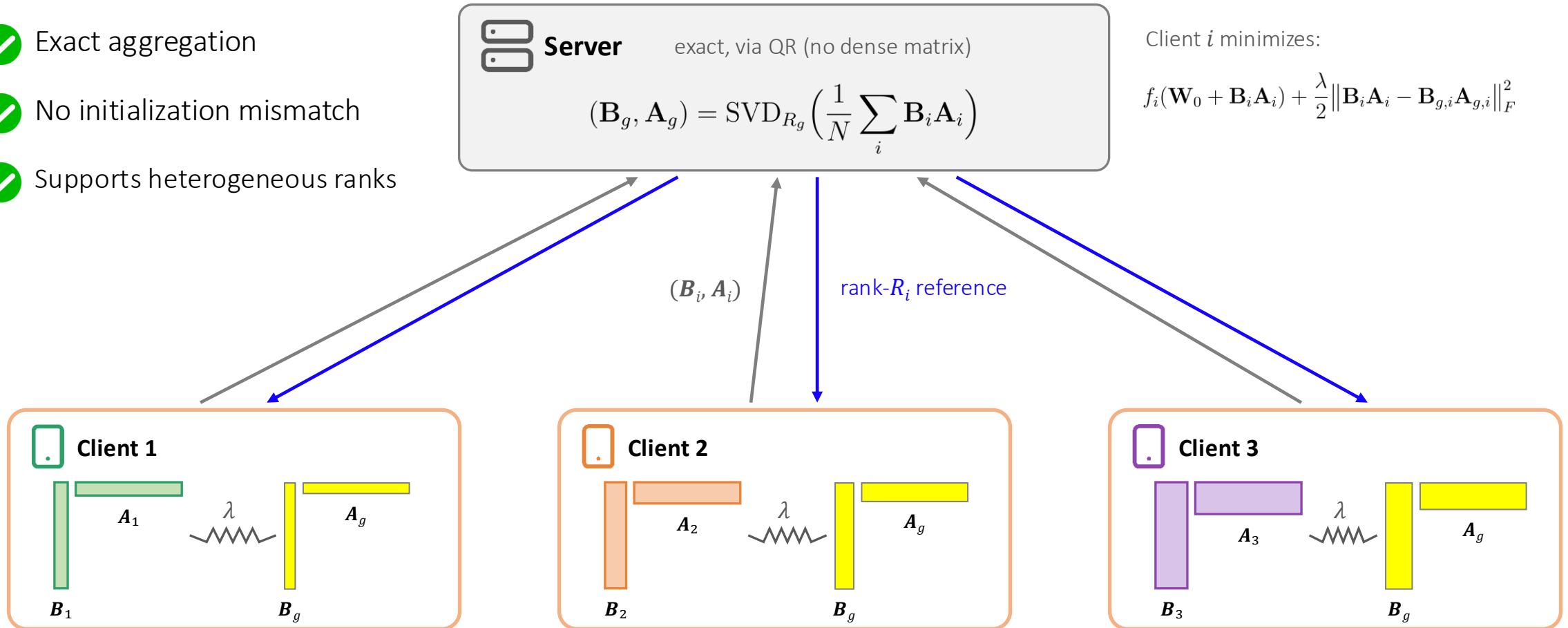
MNLI 0.906

Same update, different factors ($2 \cdot 1 = 1 \cdot 2$): factor pulls punish this harmless difference, the product pull does not

FedPA-LoRA: Proposed

FedPA-LoRA: Keep Local Factors, Aggregate Products (Proposed)

- ✓ Exact aggregation
- ✓ No initialization mismatch
- ✓ Supports heterogeneous ranks



Clients keep their own factors; λ pulls $\mathbf{B}_i \mathbf{A}_i$ toward the reference, never overwriting them

Two ranks · r_i = local rank (computation + uplink) · R_i = communication rank of the reference (downlink) — decoupled, so budgets are set per client

Experimental Setup

	Natural language understanding	Natural language generation
Model	RoBERTa-Large	Llama 3-8B
Tasks	GLUE: SST-2, QNLI, QQP, RTE, MNLI	GSM8K; HumanEval (fine-tuned on CodeSearchNet)
Data	Non-IID via Dirichlet $\beta = 0.5$ ($\beta \in \{100, 1, 0.5, 0.1\}$ in analyses)	GSM8K: 3 IID clients; HumanEval: 6 clients split by programming language
Ranks	Homogeneous: $N = 10, r = 4$ Heterogeneous: $N = 20, r_i \in \{2, 4, 16\}$	Homogeneous: $r = 8$ Heterogeneous: $r_i \in \{2, 4, 16\}$
Training	250 rounds $\times$ 20 local steps	200 rounds $\times$ 30 local steps

Baselines: FedIT, FlexLoRA, FFA-LoRA, RoLoRA, FedRot-LoRA, FedDPA-LoRA, FedSA-LoRA, HetLoRA, Ravan, Fed-PLoRA

Also: ViT-B/16 on CIFAR-10/100 ($N = 10, r = 4, \beta = 0.1$)

Natural Language Understanding (GLUE)

RoBERTa-Large, $\beta = 0.5$ (homogeneous: $N = 10, r = 4$; heterogeneous: $N = 20$)

Setting	Method	SST-2	QNLI	QQP	RTE	MNLI	Average
Homo.	FedIT	0.956 ± 0.000	0.891 ± 0.046	0.861 ± 0.001	0.696 ± 0.018	0.858 ± 0.001	0.8524
	FlexLoRA	0.956 ± 0.001	0.642 ± 0.100	0.848 ± 0.010	0.742 ± 0.061	0.858 ± 0.002	0.8092
	FFA-LoRA	0.946 ± 0.001	0.890 ± 0.010	0.848 ± 0.002	0.611 ± 0.021	0.840 ± 0.001	0.8270
	RoLoRA	0.957 ± 0.002	0.882 ± 0.046	0.845 ± 0.024	0.794 ± 0.022	0.852 ± 0.002	<u>0.8660</u>
	FedRot-LoRA	0.954 ± 0.002	0.908 ± 0.004	0.849 ± 0.002	0.714 ± 0.038	0.858 ± 0.002	0.8566
	FedDPA-LoRA	0.957 ± 0.001	0.851 ± 0.062	0.811 ± 0.045	0.612 ± 0.105	0.879 ± 0.002	0.8220
	FedSA-LoRA	0.959 ± 0.002	0.905 ± 0.006	0.814 ± 0.025	0.766 ± 0.009	0.880 ± 0.001	0.8648
	FedPA-LoRA	0.969 ± 0.001	0.942 ± 0.003	0.884 ± 0.001	0.871 ± 0.005	0.907 ± 0.002	0.9146
Hetero.	FlexLoRA	0.850 ± 0.015	0.798 ± 0.098	0.761 ± 0.033	0.570 ± 0.011	0.832 ± 0.031	0.7622
	HetLoRA	0.957 ± 0.001	0.919 ± 0.003	0.850 ± 0.002	0.671 ± 0.039	0.875 ± 0.000	<u>0.8544</u>
	Ravan	0.953 ± 0.001	0.883 ± 0.040	0.851 ± 0.003	0.712 ± 0.016	0.866 ± 0.003	0.8530
	Fed-PLoRA	0.932 ± 0.027	0.712 ± 0.137	0.811 ± 0.006	0.588 ± 0.013	0.839 ± 0.004	0.7764
	FedPA-LoRA	0.966 ± 0.003	0.959 ± 0.001	0.917 ± 0.009	0.860 ± 0.010	0.911 ± 0.003	0.9226

- ✓ Homogeneous ranks: +4.86 pp average over the best baseline (RoLoRA)
- ✓ Heterogeneous ranks: +6.82 pp average over the best baseline (HetLoRA)

Natural Language Generation (Llama 3-8B)

GSM8K: 3 IID clients; HumanEval: 6 clients split by programming language

Homogeneous ranks ($r = 8$)

Method	GSM8K (Acc.)	HumanEval (pass@1)
FedIT	0.4316 ± 0.003	0.4020 ± 0.002
FlexLoRA	0.4460 ± 0.004	0.3873 ± 0.010
FFA-LoRA	0.4316 ± 0.003	0.3902 ± 0.008
RoLoRA	0.4233 ± 0.016	0.3751 ± 0.003
FedRot-LoRA	0.4412 ± 0.014	0.4016 ± 0.007
FedPA-LoRA	0.4672 ± 0.006	0.4121 ± 0.002

Heterogeneous ranks (three resource groups)

Method	GSM8K (Acc.)	HumanEval (pass@1)
FlexLoRA	0.4488 ± 0.013	0.4012 ± 0.070
HetLoRA	0.4230 ± 0.001	0.3894 ± 0.007
Ravan	0.4735 ± 0.005	0.3959 ± 0.009
Fed-PLoRA	0.4319 ± 0.018	0.4016 ± 0.021
FedPA-LoRA	0.5018 ± 0.017	0.4048 ± 0.001

- ✓ Best on GSM8K and HumanEval in both settings
- ✓ Heterogeneous: each client receives only its rank- R_i reference

Client-Side Cost per Round

Average client wall-clock time per round (seconds)

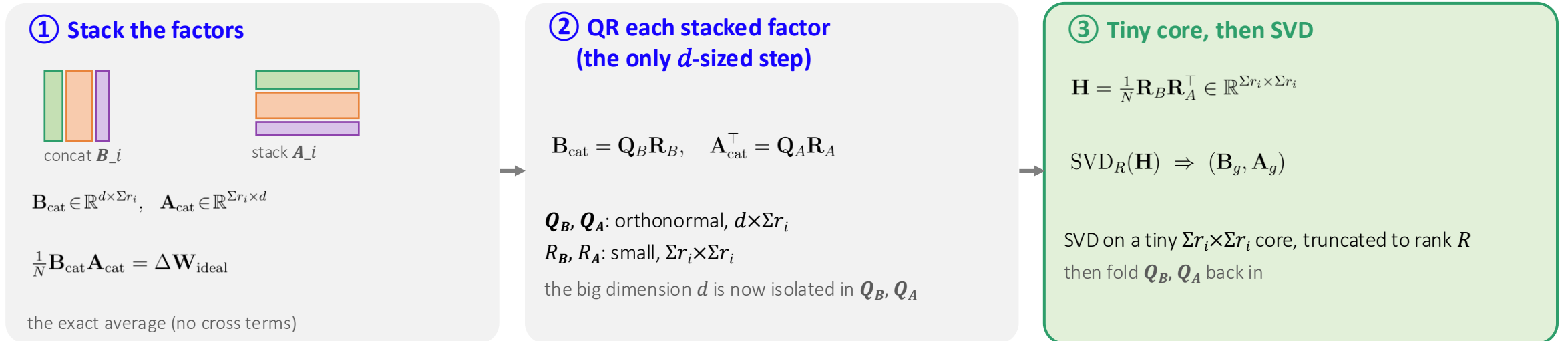
Local Update Strategy	Representative Methods	RoBERTa-Large	Llama 3-8B
Single-Factor LoRA Update	FFA-LoRA, RoLoRA	9.1388	3.8015
Full LoRA Update	FedIT, FlexLoRA, FedSA-LoRA	9.2053	3.8324
LoRA Update + Alignment	FedRot-LoRA	9.2121	3.9323
Global and Local LoRA Update	FedDPA-LoRA	10.3431	3.8371
Product-Regularized LoRA Update	FedPA-LoRA	9.4603	5.1343

- ✓ Product pull adds +2.8% on RoBERTa-Large and +34% (~1.3 s) on Llama 3-8B over a full LoRA update

Thank you!
Questions?

Server: Exact Aggregation Without Dense Matrices

Reconstruct the rank- R global adapter from the exact average — without ever forming the dense $d \times d$ matrix



✓ Only the two QRs touch dimension d ; the SVD is on a $\sum r_i \times \sum r_i$ core — never a $d \times d$ matrix

✓ Same result as a dense SVD of the average: 0.12 s vs 152 s per round (Llama 3-8B)

Two ranks (paper): r_i = local rank — computation + **uplink** · R = **communication rank** — downlink reference returned to clients ($R \leq \sum r_i$)

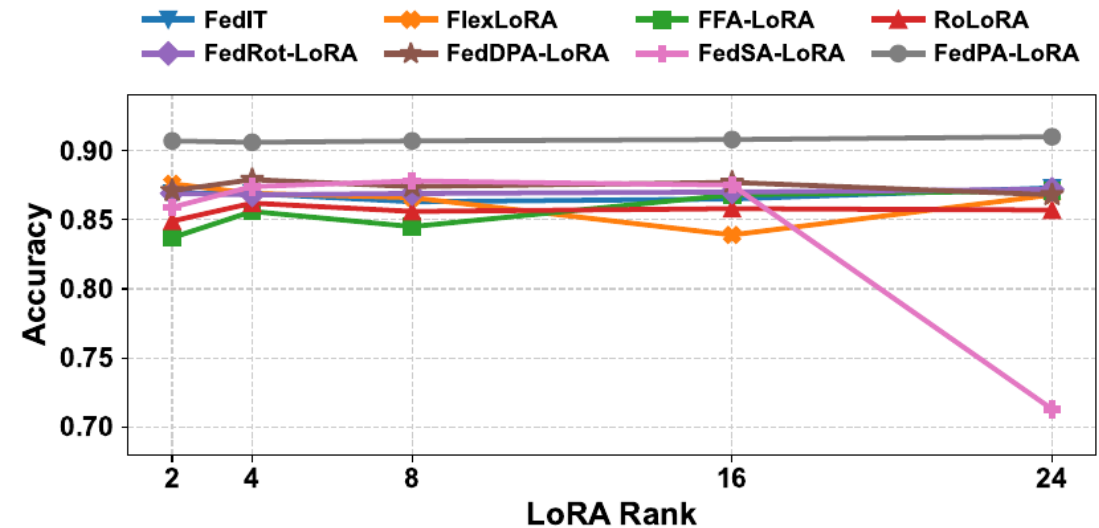
Number of Clients and LoRA Rank

MNLI accuracy, $\beta = 0.5$

Number of clients N ($r = 4$)

Method	$N = 3$	$N = 10$	$N = 50$
FedIT	0.869 ± 0.001	0.858 ± 0.001	0.877 ± 0.001
FlexLoRA	0.869 ± 0.002	0.858 ± 0.002	0.828 ± 0.025
FFA-LoRA	0.856 ± 0.002	0.840 ± 0.001	0.860 ± 0.001
RoLoRA	0.862 ± 0.000	0.852 ± 0.002	0.882 ± 0.002
FedRot-LoRA	0.868 ± 0.002	0.858 ± 0.002	0.878 ± 0.001
FedDPA-LoRA	0.879 ± 0.002	0.879 ± 0.002	0.883 ± 0.014
FedSA-LoRA	0.874 ± 0.002	0.880 ± 0.001	0.891 ± 0.000
FedPA-LoRA	0.906 ± 0.003	0.907 ± 0.002	0.916 ± 0.001

LoRA rank r ($N = 3$)



✓ Best for $N = 3$ to 50 and for every rank $r = 2$ to 24

Partial Participation and Vision

MNLI ($r = 4, \beta = 0.5$): clients N , participation rate ρ

Method	$N = 50$				$N = 100$
	$\rho = 0.1$	$\rho = 0.3$	$\rho = 0.5$	$\rho = 1.0$	$\rho = 0.3$
FedIT	0.863±0.002	0.879±0.001	0.878±0.001	0.877±0.001	0.878±0.001
FlexLoRA	0.737±0.176	0.841±0.012	0.840±0.015	0.828±0.025	0.835±0.029
FFA-LoRA	0.858±0.003	0.861±0.002	0.861±0.001	0.860±0.001	0.860±0.002
RoLoRA	0.858±0.020	0.878±0.001	0.883±0.002	0.882±0.002	0.881±0.007
FedRot-LoRA	0.876±0.003	0.878±0.001	0.879±0.002	0.878±0.001	0.878±0.001
FedDPA-LoRA	0.875±0.001	<u>0.889±0.002</u>	<u>0.890±0.002</u>	0.883±0.014	0.879±0.011
FedSA-LoRA	<u>0.877±0.003</u>	<u>0.889±0.001</u>	0.889±0.002	<u>0.891±0.000</u>	<u>0.891±0.001</u>
FedPA-LoRA	0.884±0.008	0.914±0.001	0.913±0.001	0.916±0.001	0.905±0.001

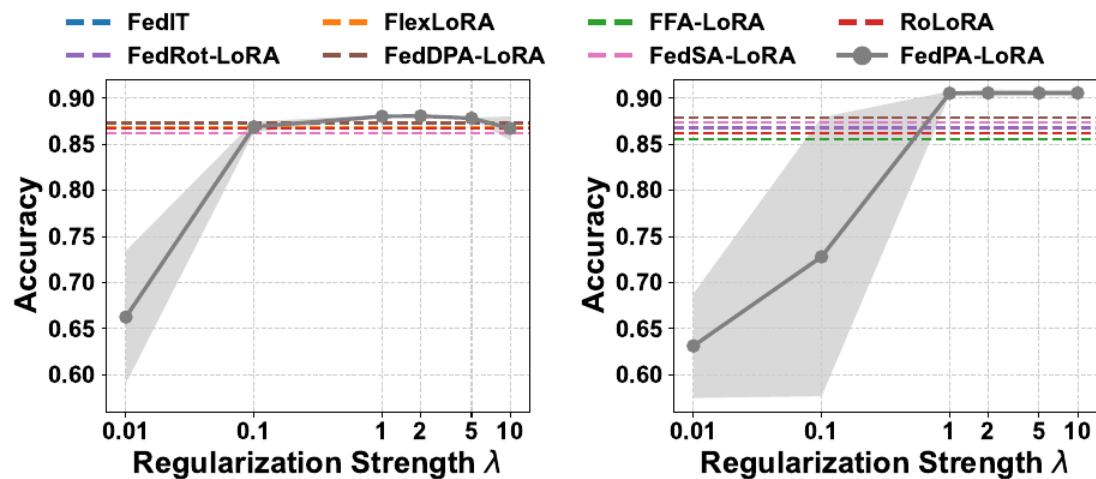
ViT-B/16 ($N = 10, r = 4, \beta = 0.1$)

Method	CIFAR-10	CIFAR-100
FedIT	0.977±0.000	0.855±0.001
FlexLoRA	0.977±0.001	0.852±0.001
FFA-LoRA	0.977±0.000	0.855±0.001
RoLoRA	0.978±0.001	0.866±0.001
FedRot-LoRA	0.978±0.001	0.855±0.001
FedDPA-LoRA	0.977±0.000	0.853±0.002
FedSA-LoRA	<u>0.984±0.001</u>	<u>0.909±0.001</u>
FedPA-LoRA	0.987±0.001	0.916±0.001

- ✓ Best even when only 10% of clients participate per round
- ✓ The gains carry over from language to vision

Sensitivity: Regularization λ and Budget R

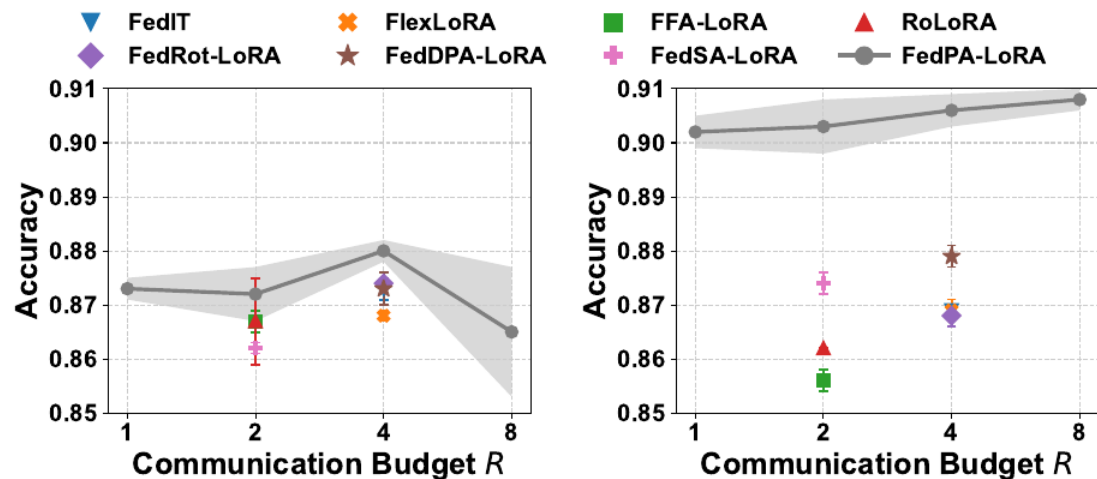
Regularization strength λ ($N = 3, r = R = 4$)



(a) IID

(b) Non-IID

Communication budget R ($N = 3, r = 4$)



(a) IID

(b) Non-IID

- ✓ Non-IID: λ from 1 to 10 is consistently strong
- ✓ Even $R = 1$ helps; a larger R helps more under non-IID data

Server-Side Cost per Round

Average server aggregation time (seconds), homogeneous ranks

Server Operation	RoBERTa-Large ($N = 10, R_g = 4$)		Llama 3-8B ($N = 6, R_g = 8$)	
	Sketch Dim. ℓ	Time (s)	Sketch Dim. ℓ	Time (s)
Two-Factor Averaging	—	0.0298	—	0.0199
Single-Factor Averaging	—	0.0206	—	0.0109
Dense Product Aggregation and SVD	—	4.7340	—	152.3957
Reduced QR and Core SVD	—	0.0840	—	0.1169
Randomized Reconstruction ($p = 0$)	4	0.0473	8	0.0596
Randomized Reconstruction ($p = 8$)	12	0.0493	16	0.0638
Randomized Reconstruction ($p = 16$)	20	0.0529	24	0.0706
Randomized Reconstruction ($p = 32$)	36	0.0615	40	0.0901

✓ Same optimum as dense SVD, [~1,300× faster](#) on Llama 3-8B (0.12 s vs. 152 s)

Server-Side Complexity (Asymptotic)

Dominant per-layer server cost, homogeneous ranks, $N r \ll d$

Method	Client Upload	Server Operation	Computation	Memory
FedIT	$\mathbf{B}_i, \mathbf{A}_i$	Two-Factor Averaging	$\mathcal{O}(Ndr)$	$\mathcal{O}(Ndr)$
FedRot-LoRA	$\tilde{\mathbf{B}}_i, \tilde{\mathbf{A}}_i$	Two-Factor Averaging	$\mathcal{O}(Ndr)$	$\mathcal{O}(Ndr)$
FedDPA-LoRA	$\mathbf{B}_{g,i}, \mathbf{A}_{g,i}$	Two-Factor Averaging	$\mathcal{O}(Ndr)$	$\mathcal{O}(Ndr)$
FFA-LoRA	$\mathbf{B}_i$	Single-Factor Averaging	$\mathcal{O}(Ndr)$	$\mathcal{O}(Ndr)$
RoLoRA	$\mathbf{B}_i$ or $\mathbf{A}_i$	Single-Factor Averaging	$\mathcal{O}(Ndr)$	$\mathcal{O}(Ndr)$
FedSA-LoRA	$\mathbf{A}_i$	Single-Factor Averaging	$\mathcal{O}(Ndr)$	$\mathcal{O}(Ndr)$
FlexLoRA	$\mathbf{B}_i, \mathbf{A}_i$	Dense Product Aggregation and SVD	$\mathcal{O}(d^3)$	$\mathcal{O}(d^2)$
FedPA-LoRA	$\mathbf{B}_i, \mathbf{A}_i$	Reduced QR and Core SVD	$\mathcal{O}(N^2dr^2)$	$\mathcal{O}(Ndr)$
FedPA-LoRA (Randomized)	$\mathbf{B}_i, \mathbf{A}_i$	Factored Randomized Reconstruction	$\mathcal{O}(Ndr^2)$	$\mathcal{O}(Ndr)$

Backup: Randomized Server Reconstruction (Sketching)

Avoid the dense aggregate with a random sketch (randomized SVD)

Idea: a random projection Ω squeezes the update's range into $\ell = R_g + p$ columns; do all SVD work in that small subspace

1 **Draw a sketch** $\Omega \in \mathbb{R}^{d \times (R_g + p)}$
shared seed; p small

2 **Sketch the range** $Y = \frac{1}{N} \sum_i B_i(A_i \Omega)$
a sum over clients

3 **Range finder** $Q = \text{orth}(Y)$
basis of the update

4 **Project + SVD** $\text{SVD}_{R_g}(Q^\top B_{\text{cat}} A_{\text{cat}}) \rightarrow (B_g, A_g)$
in the sketched subspace

- ✓ Only Step 2 touches client data, and it is a **sum over clients** → fits secure aggregation
- ✓ Accuracy holds across sketch sizes $\ell = R_g + p$ (right)

Cost (per layer)

- ✓ Exact (QR + core SVD): $O(N^2 d r^2)$
- ✓ Randomized: $O(N d r^2)$
- ✓ Error bound $\propto \sigma_{R_{\{g\}} + 1}$ (Halko et al., 2011)
- ✓ $R_g + p = r_{\text{tot}}$ recovers the exact result

